

后，动态链接器通过执行下面的重定位完成链接任务：

- 重定位 `libc.so` 的文本和数据到某个内存段。
- 重定位 `libvector.so` 的文本和数据到另一个内存段。
- 重定位 `prog21` 中所有对由 `libc.so` 和 `libvector.so` 定义的符号的引用。

最后，动态链接器将控制传递给应用程序。从这个时刻开始，共享库的位置就固定了，并且在程序执行的过程中都不会改变。

7.11 从应用程序中加载和链接共享库

到目前为止，我们已经讨论了在应用程序被加载后执行前时，动态链接器加载和链接共享库的情景。然而，应用程序还可能在它运行时要求动态链接器加载和链接某个共享库，而无需在编译时将那些库链接到应用中。

动态链接是一项强大有用的技术。下面是一些现实世界中的例子：

- 分发软件。微软 Windows 应用的开发者常常利用共享库来分发软件更新。他们生成一个共享库的新版本，然后用户可以下载，并用它替代当前的版本。下一次他们运行应用程序时，应用将自动链接和加载新的共享库。
- 构建高性能 Web 服务器。许多 Web 服务器生成动态内容，比如个性化的 Web 页面、账户余额和广告标语。早期的 Web 服务器通过使用 `fork` 和 `execve` 创建一个子进程，并在该子进程的上下文中运行 CGI 程序来生成动态内容。然而，现代高性能的 Web 服务器可以使用基于动态链接的更有效和完善的方法来生成动态内容。

其思路是将每个生成动态内容的函数打包在共享库中。当一个来自 Web 浏览器的请求到达时，服务器动态地加载和链接适当的函数，然后直接调用它，而不是使用 `fork` 和 `execve` 在子进程的上下文中运行函数。函数会一直缓存在服务器的地址空间中，所以只要一个简单的函数调用的开销就可以处理随后的请求了。这对一个繁忙的网站来说是有很大影响的。更进一步地说，在运行时无需停止服务器，就可以更新已存在的函数，以及添加新的函数。

Linux 系统为动态链接器提供了一个简单的接口，允许应用程序在运行时加载和链接共享库。

```
#include <dlfcn.h>
```

```
void *dlopen(const char *filename, int flag);
```

返回：若成功则为指向句柄的指针，若出错则为 NULL。

`dlopen` 函数加载和链接共享库 `filename`。用已用带 `RTLD_GLOBAL` 选项打开了的库解析 `filename` 中的外部符号。如果当前可执行文件是带 `-rdynamic` 选项编译的，那么对符号解析而言，它的全局符号也是可用的。`flag` 参数必须要么包括 `RTLD_NOW`，该标志告诉链接器立即解析对外部符号的引用，要么包括 `RTLD_LAZY` 标志，该标志指示链接器推迟符号解析直到执行来自库中的代码。这两个值中的任意一个都可以和 `RTLD_GLOBAL` 标志取或。

```
#include <dlfcn.h>
```

```
void *dlsym(void *handle, char *symbol);
```

返回：若成功则为指向符号的指针，若出错则为 NULL。

`dlsym` 函数的输入是一个指向前面已经打开了的共享库的句柄和一个 `symbol` 名字，如果该符号存在，就返回符号的地址，否则返回 `NULL`。

```
#include <dlfcn.h>

int dlclose (void *handle);
```

返回：若成功则为 0，若出错则为 -1。

如果没有其他共享库还在使用这个共享库，`dlclose` 函数就卸载该共享库。

```
#include <dlfcn.h>

const char *dlerror(void);
```

返回：如果前面对 `dlopen`、`dlsym` 或 `dlclose` 的调用失败，则为错误消息，如果前面的调用成功，则为 `NULL`。

`dlerror` 函数返回一个字符串，它描述的是调用 `dlopen`、`dlsym` 或者 `dlclose` 函数时发生的最近的错误，如果没有错误发生，就返回 `NULL`。

图 7-17 展示了如何利用这个接口动态链接我们的 `libvector.so` 共享库，然后调用它的 `addvec` 例程。要编译这个程序，我们将以下面的方式调用 GCC：

```
linux> gcc -rdynamic -o prog2r dll.c -ldl
```

code/link/dll.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <dlfcn.h>
4
5  int x[2] = {1, 2};
6  int y[2] = {3, 4};
7  int z[2];
8
9  int main()
10 {
11     void *handle;
12     void (*addvec)(int *, int *, int *, int);
13     char *error;
14
15     /* Dynamically load the shared library containing addvec() */
16     handle = dlopen("./libvector.so", RTLD_LAZY);
17     if (!handle) {
18         fprintf(stderr, "%s\n", dlerror());
19         exit(1);
20     }
21
22     /* Get a pointer to the addvec() function we just loaded */
23     addvec = dlsym(handle, "addvec");
24     if ((error = dlerror()) != NULL) {
25         fprintf(stderr, "%s\n", error);
```

图 7-17 示例程序 3。在运行时动态加载和链接共享库 `libvector.so`


```

26         exit(1);
27     }
28
29     /* Now we can call addvec() just like any other function */
30     addvec(x, y, z, 2);
31     printf("z = [%d %d]\n", z[0], z[1]);
32
33     /* Unload the shared library */
34     if (dlclose(handle) < 0) {
35         fprintf(stderr, "%s\n", dlerror());
36         exit(1);
37     }
38     return 0;
39 }

```

code/link/dll.c

图 7-17 (续)

旁注 共享库和 Java 本地接口

Java 定义了一个标准调用规则，叫做 Java 本地接口 (Java Native Interface, JNI)，它允许 Java 程序调用“本地的”C 和 C++ 函数。JNI 的基本思想是将本地 C 函数(如 foo)编译到一个共享库中(如 foo.so)。当一个正在运行的 Java 程序试图调用函数 foo 时，Java 解释器利用 dlopen 接口(或者与其类似的接口)动态链接和加载 foo.so，然后再调用 foo。

7.12 位置无关代码

共享库的一个主要目的就是允许多个正在运行的进程共享内存中相同的库代码，因而节约宝贵的内存资源。那么，多个进程是如何共享程序的一个副本的呢？一种方法是给每个共享库分配一个事先预备的专用的地址空间片，然后要求加载器总是在这个地址加载共享库。虽然这种方法很简单，但是它也造成了一些严重的问题。它对地址空间的使用效率不高，因为即使一个进程不使用这个库，那部分空间还是会被分配出来。它也难以管理。我们必须保证没有片会重叠。每次当一个库修改了之后，我们必须确认已分配给它的片还适合它的大小。如果不适合了，必须找一个新的片。并且，如果创建了一个新的库，我们还必须为它寻找空间。随着时间的进展，假设在一个系统中有了成百个库和库的各个版本库，就很难避免地址空间分裂成大量小的、未使用而又不再生能使用的小洞。更糟的是，对每个系统而言，库在内存中的分配都是不同的，这就引起了更多令人头痛的管理问题。

要避免这些问题，现代系统以这样一种方式编译共享模块的代码段，使得可以把它们加载到内存的任何位置而无需链接器修改。使用这种方法，无限多个进程可以共享一个共享模块的代码段的单一副本。(当然，每个进程仍然会有它自己的读/写数据块。)

可以加载而无需重定位的代码称为位置无关代码(Position-Independent Code, PIC)。用户对 GCC 使用 -fpic 选项指示 GNU 编译系统生成 PIC 代码。共享库的编译必须总是使用该选项。

在一个 x86-64 系统中，对同一个目标模块中符号的引用是不需要特殊处理使之成为 PIC。可以用 PC 相对寻址来编译这些引用，构造目标文件时由静态链接器重定位。然而，对共享模块定义的外部过程和对全局变量的引用需要一些特殊的技巧，接下来我们会谈到。